

Assignment 5
CAP 6419 - 3D Computer Vision
Dr. Hassan Foroosh
Dept. of Computer Science

For this assignment you are required to implement the linear triangulation method described in the textbook (Hartley-Zisserman, Multiple View Geometry, Chapter 11 in the 1st Edition or Chapter 12 in the 2nd Edition).

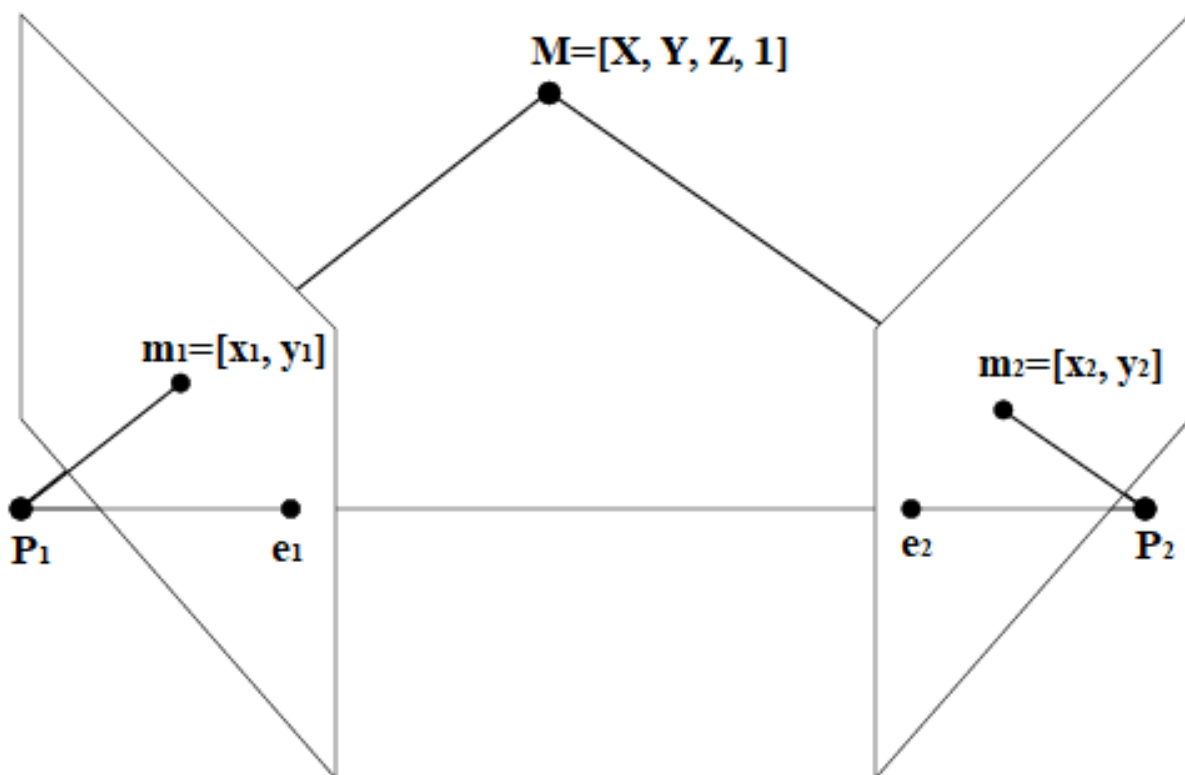


Figure 1 – Linear back-projection or triangulation of matching points in a stereo pair.

The basic idea is depicted in Figure 1. Given a pair of images as a stereo view of a scene, we can use matching points to triangulate the corresponding 3D point, and therefore reconstruct a point cloud model of the 3D scene. Of course, the density of the point cloud model depends on the number of matching points.

A linear solution to this problem, as discussed in the class, is as follows (all notations correspond to Figure 1 above):

- Step 1: Find a set of feature points in each image, and match the corresponding points to form matching point-pairs $\mathbf{m}_1 = [x_1, y_1] \leftrightarrow \mathbf{m}_2 = [x_2, y_2]$
- Step 2: Estimate the fundamental matrix $\mathbf{F}$ from the matching points
- Step 3: Assuming that the intrinsic calibration matrix $\mathbf{K}$ is given and is fixed (not changing between the two views), estimate the camera projection matrices $\mathbf{P}_1$ and $\mathbf{P}_2$. This requires estimating the camera poses, i.e. estimating the translation vector $\mathbf{t}$ and the relative rotation matrix $\mathbf{R}$ of the second view with respect to the first view.
 - If you attach the world coordinate frame to the first camera, then the two camera projection matrices will be:

$$\mathbf{P}_1 = \mathbf{K}[\mathbf{I} \mid \mathbf{0}]$$

$$\mathbf{P}_2 = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$$

Matlab has a built-in function called `relativeCameraPose` for estimating the relative camera pose from a set of point correspondences, the intrinsic calibration matrix $\mathbf{K}$ and the fundamental matrix $\mathbf{F}$.

- Step 4: Given the matching image points $\mathbf{m}_1 = [x_1, y_1] \leftrightarrow \mathbf{m}_2 = [x_2, y_2]$ and the two 3×4 camera matrices $\mathbf{P}_1$ and $\mathbf{P}_2$, for each matching pair of image points build the matrix $\mathbf{A}$ as follows, so that $\mathbf{A}\mathbf{M} = \mathbf{0}$:

$$\mathbf{A} = \begin{bmatrix} x_1 \mathbf{P}_1^{3T} - \mathbf{P}_1^{1T} \\ y_1 \mathbf{P}_1^{3T} - \mathbf{P}_1^{2T} \\ x_2 \mathbf{P}_2^{3T} - \mathbf{P}_2^{1T} \\ y_2 \mathbf{P}_2^{3T} - \mathbf{P}_2^{2T} \end{bmatrix}$$

where $\mathbf{P}_i^j$ is the j^{th} row of the camera projection matrix $\mathbf{P}_i, i = 1, 2$, and the transposition is to make sure it is a row vector. So, for instance, $\mathbf{P}_1^{3T}$ is the 3rd row of the first camera projection matrix. Therefore, $\mathbf{A}$ is a 4×4 matrix.

- Step 5: Use singular value decomposition to find 3D point $\mathbf{M}$, corresponding to the matching image points $\mathbf{m}_1$ and $\mathbf{m}_2$.
- Step 6: Repeat from Step 4 for all matching image points to create a 3D point cloud, i.e. a set of 3D points with known X, Y, Z coordinates.
- Step 7: Render the point cloud.

What to submit:

- 1- Write a Matlab program called `linbackproj.m` that implements the linear back-projection method described above. Incorporate it as part of a Matlab function or Matlab script that performs point matching and loops through all point correspondences to determine the corresponding 3D points. (40 pts.)
- 2- I have provided four sets of stereo test images in four separate folders. Each folder contains a pair of images called `im1.png` and `im2.png` and a matlab script called `calib.m`.

To run and test your `linbackproj.m` program you need to first run the script `calib.m` to load the two images and also the calibration parameters. Test your completed code on each pair and record your output results to include in your report. Since the image point

correspondences are sparse, it might be difficult to visually inspect your results to see if your algorithm works well. You need to display and include in your report 3 images: the image pair with matching points, the rendered 3D point cloud that your implementation of the algorithm generates, and a reference 3D point cloud generated by a built-in Matlab function called `triangulate`. This will allow you to compare your point cloud with the one produced by Matlab as a sanity check for your implementation of the linear backprojection algorithm. (40 pts.)

For the image pair in the folder called `globe` you can actually see the shape of the globe, so that will give you a visual inspection.

- 3- Compute the signal-to-noise ratio of your 3D point cloud by taking the one by Matlab as the ground-truth and include it in your report. If your algorithms are implemented correctly you should get a very high SNR (of the order of 50dB or above) – I got over 80dB. Record these SNR values for your report.
- 4- Submit a report on the project including a brief description of your implementation, all input test images, your test results both visual rendering/inspections and the SNR values, and any conclusions or thoughts. (20 pts.)